



Cấu trúc dữ liệu & giải thuật

Data Structure and Algorithms

Chương 4: NGĂN XẾP & HÀNG ĐỢI

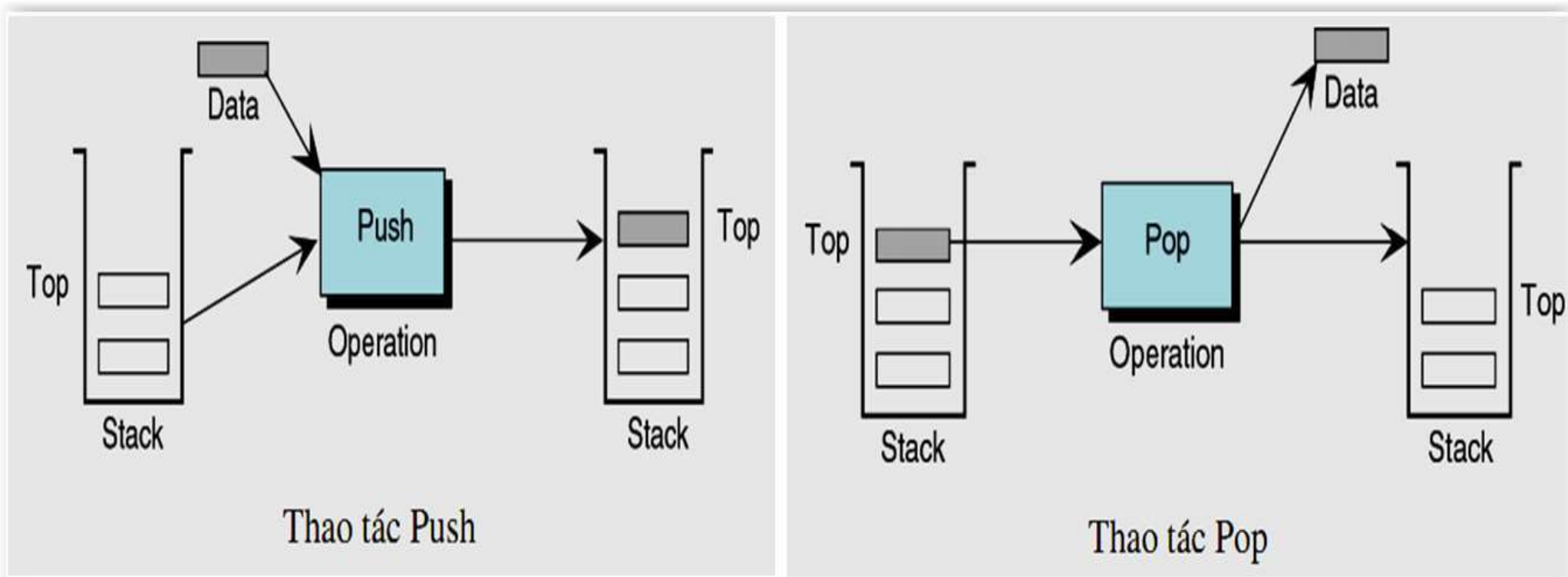
TS. Võ Văn Tuấn Dũng

Trường Đại học Tài nguyên Môi trường TP. HCM

4.1. NGĂN XẾP (STACK)

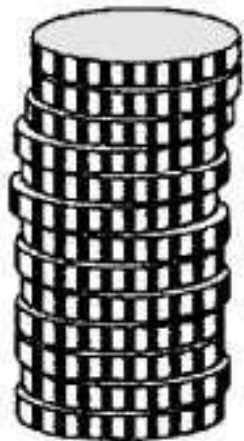
4.1.1. Khái niệm:

Stack là dạng đặc biệt của danh sách tuyến tính, trong đó các đối tượng được **nhập vào** (*push*) và **lấy ra** (*pop*) chỉ từ một đầu được gọi là **đỉnh** (*top*) của danh sách.

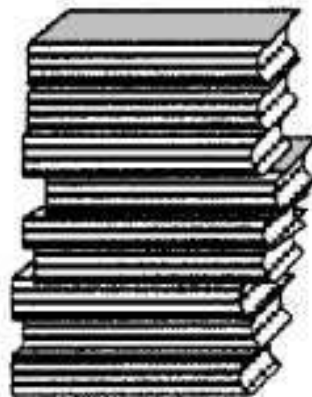


4.1. NGĂN XẾP (STACK)

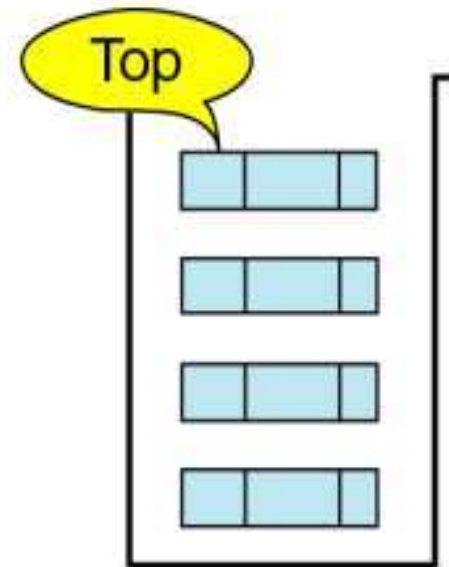
Nguyên tắc hoạt động: Stack hoạt động theo cơ chế: “**vào sau ra trước**” (*last in first out*). Do đó **stack** còn được gọi là **danh sách kiểu LIFO**. Stack có thể rỗng, nghĩa là không có phần tử nào.



Stack of coins



Stack of books



Computer stack

4.1. NGĂN XẾP (STACK)

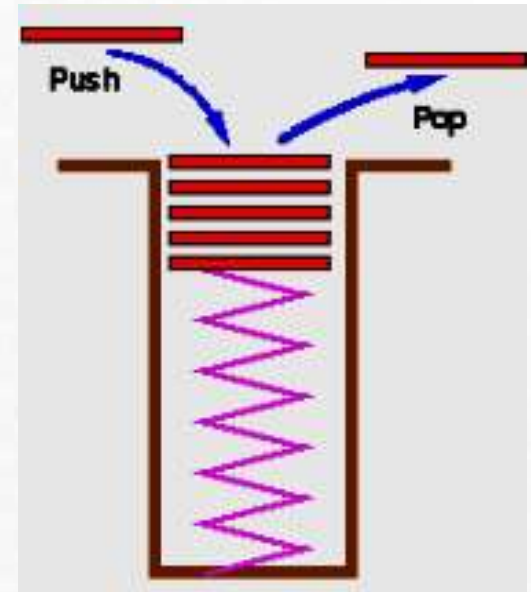
Stack gồm nhiều phần tử có cùng kiểu dữ liệu, phần tử trên cùng **stack** luôn luôn có một con trỏ chỉ tới ta gọi là **Stack Pointer** (ký hiệu: **sp**).

Ta xét hai cách tổ chức stack:

- Sử dụng mảng;
- Sử dụng danh sách liên kết.

★ Trên stack có 2 phép toán chính:

- **Push** : thêm một phần tử vào đầu **stack**
- **Pop** : xóa một phần tử khỏi **stack**, trả cho chương trình gọi giá trị của phần tử vừa xóa.



4.1. NGĂN XẾP (STACK)

4.1.2. Stack dùng mảng:

1. Khai báo: Ta khai báo biến **st** có kiểu cấu trúc **stack** như sau:

```
#define STACKSIZE 100
#define TRUE 1
#define FALSE 0
struct stack
{
    int sp;
    int nodes[STACKSIZE];
} st;
```

4.1. NGĂN XẾP (STACK)

2. Các phép toán:

a) **Phép toán push**: Thêm một phần tử có giá trị **x** vào đầu stack.

```
void push (struct stack &st, int x)
```

```
{ if(st.sp == STACKSIZE-1)  
    { cout<< "stack bi day";  
      exit(1);  
    }  
    else st.nodes[++(st.sp)] = x;  
}
```

4.1. NGĂN XẾP (STACK)

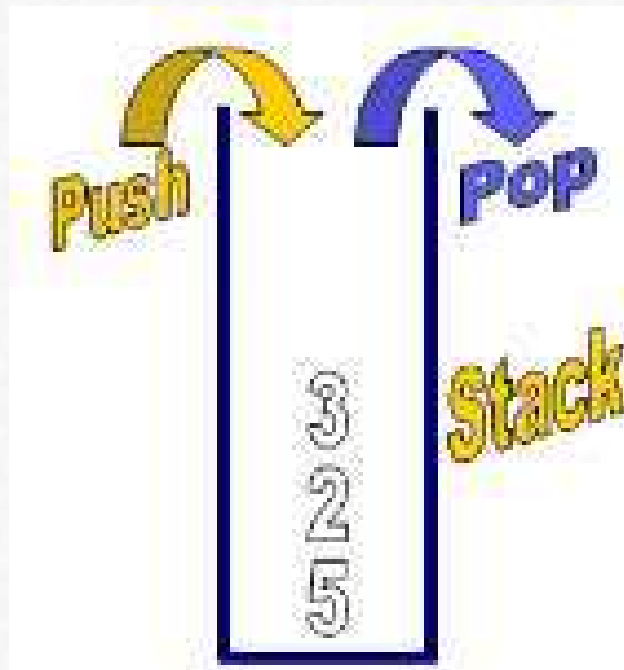
b) Phép toán pop: Loại bỏ phần tử khỏi **stack** và trả về giá trị của phần tử vừa xóa; trước khi xóa, ta phải kiểm tra **stack** có khác rỗng hay không.

```
int pop(struct stack &st)
{ if(empty(st))
    { cout << "stack bi rong");
      exit(1);
    }
  return(st.nodes[(st.sp)--]);
}
```

4.1. NGĂN XẾP (STACK)

4.1.3. Ứng dụng của Stack dùng mảng:

- Stack thường được dùng trong các bài toán có cơ chế LIFO (*vào sau ra trước*).
- Stack cũng được dùng trong các bài toán gỡ đệ qui (*chuyển một giải thuật đệ qui thành giải thuật không đệ qui*).



4.1. NGĂN XẾP (STACK)

Ví dụ: Viết chương trình đổi số nguyên không âm ở hệ thập phân sang số ở hệ nhị phân.

Nhắc lại: Quy tắc đổi một số N từ hệ 10 sang hệ cơ số b ($b \neq 10$): Lấy N chia cho b cho đến khi phần thương của phép chia bằng 0, số đổi được chính là các phần dư của phép chia theo thứ tự ngược lại.

Ví dụ: Đổi số 6 (hệ thập phân) sang hệ nhị phân.

Trong cách chuyển đổi này các số dư được tạo ra sau lại được hiển thị trước. Cơ chế sắp xếp này phù hợp với cơ chế hoạt động của stack. Do đó, người ta sử dụng 1 stack để lưu trữ các số dư, sau đó lại lấy các số này từ stack ra để hiển thị thành mã số nhị phân.

$$\begin{array}{r|l} 6 & 2 \\ \hline 0 & 3 \\ \hline & 1 \\ \hline & 1 \\ \hline & 0 \end{array}$$

$$\Rightarrow 6_{10} = 110_2.$$

[vd-stack-doiHe10-He2.docx](#)

4.1. NGĂN XẾP (STACK)

4.1.4. Cài đặt stack với danh sách liên kết:

Cài đặt Stack bằng mảng có hạn chế là số lượng phần tử phải khai báo trước. Để khắc phục nhược điểm này, ta sẽ xây dựng Stack với cấu trúc dữ liệu là DSLK đơn.

1. Khai báo: Ta khai báo biến **sp** (Stack Pointer) là con trỏ chỉ đến một danh sách là Stack, mỗi phần tử trong Stack là 1 số nguyên như sau:

```
struct node
{
    int info;
    struct node *next ;
};
typedef struct node *Stack;
Stack sp;
```

4.1. NGĂN XẾP (STACK)

2. Khởi tạo stack:

`sp = NULL;`

Lưu ý:

- Với Stack là DSLK, ta chỉ thực hiện các phép toán ở đầu danh sách.
- Không có trường hợp Stack đầy; Stack rỗng khi `sp = NULL`.

4.1. NGĂN XẾP (STACK)

3. Các phép toán:

a) Phép thêm vào (push): Thêm một phần tử có giá trị **x** vào đầu Stack.

void push(Stack &sp, int x)

```
{ Stack p;  
  p = (Stack) malloc(sizeof(struct node));  
  p->info = x; p->next = sp;  
  sp = p;  
}
```

[push.jpg](#)

b) Phép loại bỏ (pop): Xóa phần tử khỏi Stack và trả cho chương trình gọi giá trị của phần tử vừa xóa.

```
int pop(Stack &sp)
{
    Stack p;
    int x;
    if(sp == NULL)
        cout<<"\nStack rong";
    else
    {
        p = sp; // nut can xoa la nut dau
        x = sp->info; sp = p->next;
        free(p); return x;
    }
}
```


4.2. HÀNG ĐỢI (QUEUE)

4.2.1. Khái niệm:

- **queue** là 1 kiểu danh sách mà phép bổ sung thực hiện ở 1 đầu, gọi là ***lối sau*** (*rear*), còn phép loại bỏ lại thực hiện ở 1 đầu khác, gọi là ***lối trước*** (*front*).
- Cơ cấu của **queue** giống như 1 hàng đợi: *hàng ô tô chờ qua phà, danh sách khách hàng mua vé tàu hỏa ...*
- Các phần tử của **queue** phải được xử lý theo thứ tự mà chúng được tạo ra, nghĩa là vào ở đầu này thì ra ở đầu kia và ***vào trước thì ra trước***. Vì vậy **queue** còn được gọi là **danh sách kiểu *FIFO*** (*first in first out*)



4.2. HÀNG ĐỢI (QUEUE)

Queue chứa các phần tử có cùng kiểu dữ liệu.

- Vị trí để loại bỏ phần tử được gọi là **front**
- Vị trí để thêm phần tử vào được gọi là **rear**

Queue cũng có thể được cài đặt bằng mảng hoặc DSLK.



4.2. HÀNG ĐỢI (QUEUE)

4.2.2. Cài đặt Queue bằng mảng:

- Giả sử hàng có n phần tử, ta có $front = 0$ và $rear = n - 1$.
- Khi xoá một phần tử **$front$ tăng lên 1**, khi thêm một phần tử **$rear$ tăng lên 1**.
⇒ hàng có khuynh hướng đi xuống, đến một lúc không thể thêm vào hàng được ($rear = maxlength - 1$) dù mảng còn nhiều chỗ trống (các vị trí trước $front$), trường hợp này ta gọi là hàng **bị tràn**.
- Trường hợp toàn bộ mảng đã chứa các phần tử của hàng ta gọi là hàng **bị đầy**.

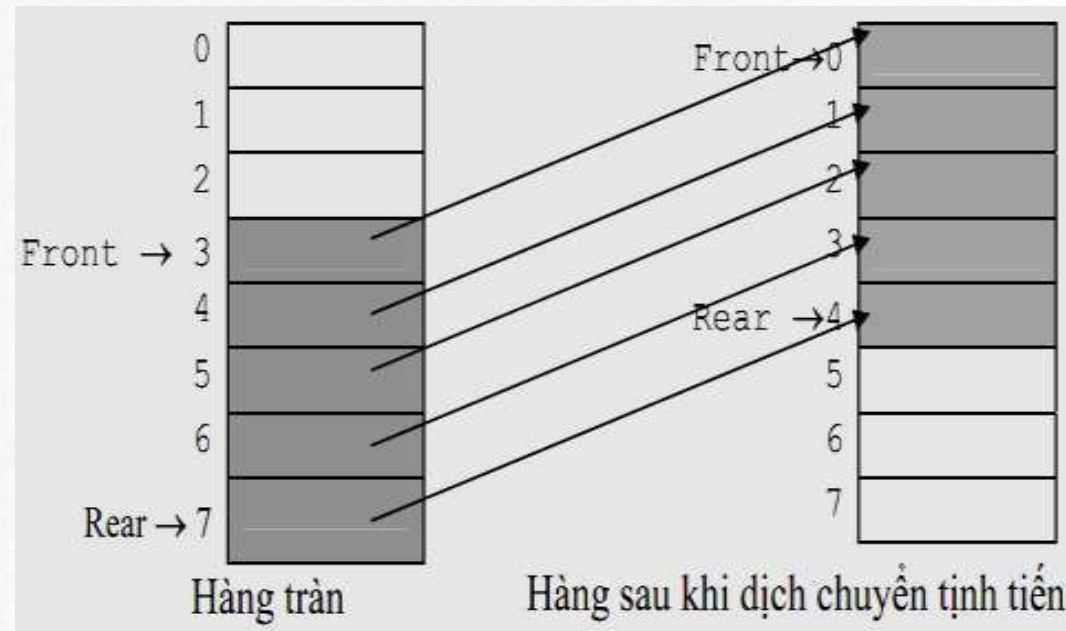
4.2. HÀNG ĐỢI (QUEUE)

1. Phép toán trên Queue: có hai phép toán chính

a) Insert_queue: Thêm một phần tử vào hàng đợi. Khi thêm ta lưu ý xem hàng đợi bị **tràn** hay bị **đầy**.

Hàng đợi bị tràn: Là trường hợp $\text{rear} = \text{QUEUESIZE} - 1 (= n)$ và $\text{front} > 0$, tức là không thể thêm phần tử mới vào cuối hàng. Để khắc phục, có thể dùng một trong 2 cách:

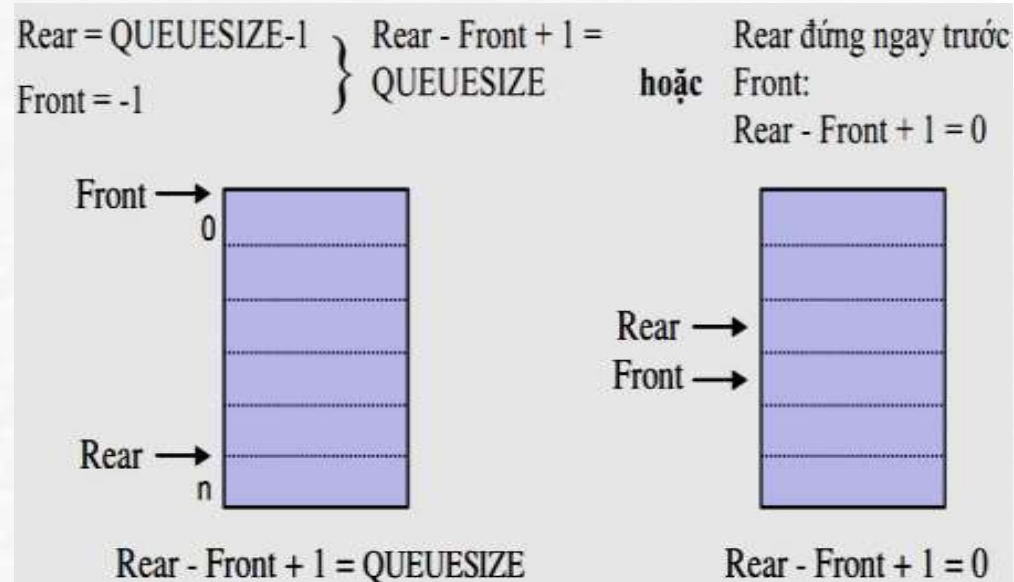
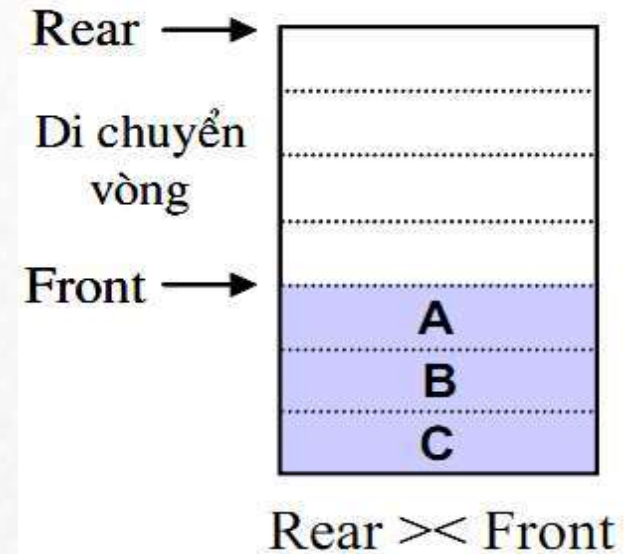
Cách 1: Di chuyển tịnh tiến từng phần tử lên để có $\text{front} = 0$ và $\text{rear} < \text{QUEUESIZE} - 1$ (trường hợp này $\text{front} \leq \text{rear}$)



4.2. HÀNG ĐỢI (QUEUE)

Cách 2: Xem mảng như là 1 vòng tròn, thêm phần tử mới vào vị trí 0 của mảng, thêm một phần tử mới nữa thì thêm vào vị trí 1 (*nếu có thể*)... Khi đó, front có thể lớn hơn rear. Cách khắc phục này gọi là dùng mảng xoay vòng.

Hàng đợi bị đầy: hàng đợi không có phần tử nào trống: $\text{front} = 0$ và $\text{rear} = n$ hoặc rear đứng ngay trước front $\Rightarrow$ nếu tiếp tục thêm vào sẽ bị mất dữ liệu.



4.2. HÀNG ĐỢI (QUEUE)

b) Delete_queue: loại bỏ phần tử khỏi Queue và trả về giá trị của phần tử vừa xóa.

Lưu ý: Phép toán ***Insert_queue*** thực hiện ở cuối hàng đợi, còn phép toán ***Delete_queue*** ở đầu hàng đợi.

2. Khởi tạo hàng đợi (tạo hàng rỗng): Lúc này ***front*** và ***rear*** không trở đến vị trí hợp lệ nào trong mảng vậy ta có thể cho ***front*** và ***rear*** đều bằng -1.

Ta khai báo biến **q** có kiểu cấu trúc Queue gồm 3 thành phần:

- **front, rear:** số nguyên chỉ đầu và cuối hàng đợi.
- **nodes:** mảng 1 chiều, mỗi phần tử của mảng là 1 phần tử trong queue.

4.2. HÀNG ĐỢI (QUEUE)

```
#define QUEUE_SIZE 100
#define TRUE 1
#define FALSE 0
struct queue
{
    int front, rear;
    int nodes[QUEUE_SIZE];
} q;
```

4.2. HÀNG ĐỢI (QUEUE)

3. Phép thêm vào: Thêm phần tử x vào hàng đợi. Khi thêm vào ta lưu ý hàng đợi bị tràn/đầy để xử lý cho thích hợp.

	0	1	2	3	4	5	6	7	8
Queue ban đầu			1	3	6	5	2		
			front = 2				rear = 6		
insert_queue(9)			1	3	6	5	2	9	
			front = 2				rear = 7		

[void Insert_queue.jpg](#)

4. Phép loại bỏ: Loại bỏ phần tử khỏi Queue và trả về giá trị của phần tử vừa xóa.

[Int Delete_queue.jpg](#)

4.2. HÀNG ĐỢI (QUEUE)

4.2.3. Cài đặt bằng DSLK:

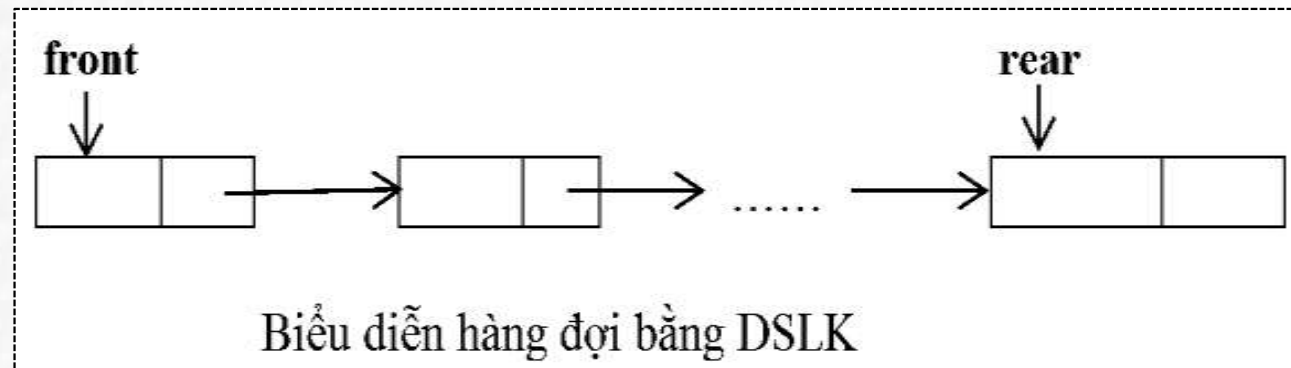
1. Khai báo: Ta khai báo biến **q** có kiểu cấu trúc Queue gồm 2 thành phần **Front**, **Rear** là con trỏ chỉ đầu và cuối hàng đợi. Mỗi phần tử của hàng đợi là một nút chứa một số nguyên.

struct node

```
{ int info;  
  struct node *next;  
};
```

```
typedef node *Hangdoi;  
struct Queue  
{ Hangdoi Front, Rear;  
};
```

```
struct Queue q;
```



4.2. HÀNG ĐỢI (QUEUE)

2. Khởi tạo hàng đợi:

`q.Front = NULL;`

3. Các phép toán trên Queue:

a) Phép thêm vào: Thêm vào cuối DSLK $\Rightarrow$ Rear thay đổi

`void Insert_queue(Queue &q, int x)`

```
{ Hangdoi p;  
  p = (Hangdoi)malloc(sizeof(struct node));  
  p->info = x;  
  if (q.Front==NULL)  
    q.Front=p;  
  else  
    q.Rear->next=p;  
  q.Rear=p; p->next=NULL;
```


4.2. HÀNG ĐỢI (QUEUE)

b) Phép loại bỏ: loại bỏ phần tử khỏi Queue và trả về giá trị của phần tử vừa xóa $\Rightarrow$ Front thay đổi.

```
int Delete_queue(Queue &q)
```

```
{   Hangdoi p;
```

```
    int x;
```

```
    if (q.Front==NULL)
```

```
        cout << "\nHàng đợi rỗng.";
```

```
    else
```

```
    {   p = q.Front; //nút cần xóa là nút đầu
```

```
        x = p->info; q.Front = p->next;
```

```
        free(p);
```

```
        return(x);
```

```
    }
```

4.2. HÀNG ĐỢI (QUEUE)

c) Minh họa các thao tác:

Thao tác	Output	Hàng đợi
Insert_queue(10)		(10)
Insert_queue(5)		(10,5)
Front()	10	
Delete_queue()		(5)
Delete_queue()		()
Front()	Lỗi: hàng đợi rỗng	()
Delete_queue()	Lỗi: hàng đợi rỗng	()
Empyt()	True	()
Insert_queue(8)		(8)

4.2. HÀNG ĐỢI (QUEUE)

4.2.4. Ứng dụng của Queue:

Ứng dụng trực tiếp:

- Danh sách xếp hàng chờ mua vé tàu xe, chờ gửi xe, chờ mượn sách ở thư viện...
- Chia sẻ các tài nguyên (ví dụ: máy in, CPU, bộ nhớ...).
- Tổ chức thực hiện đa chương trình.
- ...

Ứng dụng gián tiếp:

- Cấu trúc dữ liệu hỗ trợ cho các thuật toán.
- Là thành phần của những cấu trúc dữ liệu khác.

[tong-quan CTDL.pptx](#)